

Part 1

unsynchronized shared counter, 5 runs

Run 1: pi=3.1418888, runtime=9119.91 ms

Run 2: pi=3.1417670, runtime=8000.15 ms

Run 3: pi=3.1418370, runtime=7615.33 ms

Run 4: pi=3.1419054, runtime=7745.57 ms

Run 5: pi=3.1416993, runtime=7466.16 ms

Part 2

synchronized shared counter vs. one thread

Single thread: pi=3.1416751, runtime=6808.58 ms

Synchronized: pi=3.1417401, runtime=13422.97 ms

Part 3

reduction benchmark (100,000,000 iterations)

Threads	Runtime (ms)	Pi value
1	13803.77	pi=3.1416974
2	15446.01	pi=3.1417529
4	14901.13	pi=3.1416730
8	15118.57	pi=3.1418228
16	14775.30	pi=3.1415877
32	14864.40	pi=3.1416709

Threads	Runtime (ms)	Speedup vs. 1 thread	Efficiency
1	13803.77	1.00x	100.00%
2	15446.01	0.89x	44.68%
4	14901.13	0.93x	23.16%
8	15118.57	0.91x	11.41%
16	14775.30	0.93x	5.84%

Questions

1) Why was 16-thread execution not twice as fast as 8 threads?

The CPU has 8 physical cores. Using 16 threads oversubscribes the available cores, so threads compete for CPU time and require additional context switching. Memory and interpreter overhead also limit scalability. In this Python implementation, the GIL prevents CPU-bound threads from executing Python code truly in parallel.

2) Why was the synchronized version slower than one thread?

Every hit requires acquiring and releasing a lock. Threads spend time waiting for access to the shared counter, and lock management adds overhead. The synchronization cost is greater than the benefit of using multiple threads for this workload.